COMPARATIVE DEEP LEARNING APPROACHES FOR NETWORK INTRUSION DETECTION

Interim Report on
Deep Learning Project
[ICT-4442]

Submitted By

Aditi Arun (Reg. No. 230911108)
Anushka Thakur (Reg. No. 230911048)



SCHOOL OF COMPUTER ENGINEERING
MANIPAL INSTITUTE OF TECHNOLOGY,
MANIPAL ACADEMY OF HIGHER EDUCATION

SEPTEMBER 2026

PART B : INTERIM REPORT (Phase 2)

Team No. and Names (with Registration Numbers):

Team No.: To be assigned
1. Aditi Arun – Registration No. 230911108
2. Anushka Thakur – Registration No. 230911048

Title of the Project (confirmed / updated from Synopsis):

Comparative Deep Learning Approaches for Network Intrusion Detection (confirmed, unchanged from the Synopsis).

GitHub Repository Link:

<GitHub repository URL>

1. Literature Review

Ten papers were reviewed, covering the dataset itself, feed-forward, recurrent, convolutional, hybrid, and attention-based detectors, and known pitfalls of the dataset. They are summarised below; full references are listed at the end of this part.

Paper (Author, Year)	Method	Dataset	Key Result	Relevance to Project
[1] Sharafaldin et al., 2018	CICFlowMeter flow-feature extraction; benchmark of KNN, RF, ID3, NB, MLP, etc.; RF-based feature ranking	CIC-IDS2017	Tree-based classifiers (ID3, RF) performed best, ≈0.98 weighted precision	Source of our dataset and its 78 flow features; defines the classical baseline to beat
[2] Shone et al., 2018	Stacked non-symmetric deep autoencoders (NDAE) for unsupervised features + Random Forest	KDD Cup '99, NSL-KDD	97.85% 5-class accuracy on KDD Cup '99; weaker on rare classes in NSL-KDD	Deep feature learning on flow statistics; highlights the rare-class problem we also face
[3] Vinayakumar et al., 2019	Deep feed-forward DNN (1–5 hidden layers) vs. classical ML; scale-hybrid-IDS-AlertNet	KDDCup99, NSL-KDD, UNSW-NB15, Kyoto, WSN-DS, CIC-IDS2017	DNNs outperformed classical classifiers across the benchmark datasets	Feed-forward DNN on CIC-IDS2017 – direct reference for our MLP baseline
[4] Yin et al., 2017	RNN-IDS: recurrent neural network classifier	NSL-KDD	83.28% binary and 81.29% 5-class accuracy on KDDTest+, above J48, NB, RF, SVM	Recurrent modelling of traffic records – reference for our LSTM
[5] Kim et al., 2016	LSTM-RNN classifier	KDD Cup '99	96.93% accuracy and 98.88% detection rate, with a slightly higher false-alarm rate	LSTM for IDS; shows why false-alarm rate must be reported alongside accuracy
[6] Wang et al., 2018 (HAST-IDS)	CNN learns spatial features of packets, LSTM learns temporal features across packets	DARPA1998, ISCX2012	Higher accuracy and detection rate, lower false-alarm rate than prior published methods	Hybrid CNN/LSTM; motivates evaluating CNN and LSTM separately under one protocol
[7] Wu & Guo, 2019 (LuNet)	Hierarchical CNN + RNN with gradually increasing granularity	NSL-KDD, UNSW-NB15	NSL-KDD binary: 99.42% DR, 0.53% FPR; UNSW-NB15 multi-class: 84.98% accuracy	CNN-based local feature extraction on flow features; emphasises low false-positive rate
[8] Yang et al., 2019	Modified density-peak clustering (MDPCA) + ensemble of sub-DBNs with fuzzy aggregation	NSL-KDD, UNSW-NB15	Higher accuracy, recall, precision and F1 than well-known methods; clustering eases class imbalance	Imbalance handling – relevant to our rare Bot / Web-attack classes
[9] Gorishniy et al., 2021	FT-Transformer (each feature becomes a token) and ResNet for tabular data	11 public tabular benchmarks	FT-Transformer outperformed ResNet on most tasks; tuned MLP/ResNet are strong baselines	Basis for our Transformer design: every flow feature is a token
[10] Engelen et al., 2021	Audit of CIC-IDS2017 traffic generation, flow construction, feature extraction and labelling	CIC-IDS2017	>25% of flows are meaningless artefacts (up to 50% in some attack classes); >20% of traces relabelled or rebuilt	Motivates our cleaning (de-duplication) and caution about near-perfect scores

2. Dataset Acquisition & Preprocessing

Description of preprocessing pipeline (cleaning, augmentation, encoding, normalization, train/val/test split):

Acquisition. We use the CIC-IDS2017 MachineLearningCVE flow files [1]: eight per-day files (Monday–Friday) containing 2,830,743 flows, each with 78 numeric flow features (durations, packet and byte counts, inter-arrival-time statistics, TCP flag counts, etc.) and one of 15 labels. Rather than a pre-cleaned Kaggle derivative, we download the original files (parquet mirror, ~270 MB, script scripts/download_data.sh) and perform every cleaning step in our own code (src/data.py), so each decision is visible and reproducible.

Pipeline (all steps in src/data.py, run with python -m src.data):

- **Cleaning:** removed 2,867 rows containing NaN or ±∞ (in Flow Bytes/s and Flow Packets/s), 594,712 exact duplicate rows, a duplicated column (Fwd Header Length) and 8 constant columns (e.g. Bwd PSH Flags and all six bulk-rate features).
- **Shortcut removal:** Destination Port was dropped. It identifies the service, not the behaviour of the traffic (e.g. port 21/22 ⇒ FTP/SSH-Patator), and would let a model 'detect' attacks without learning traffic patterns.
- **Label encoding:** the 15 raw labels were grouped into 7 classes – Benign, DoS (Hulk, GoldenEye, slowloris, Slowhttptest), DDoS, PortScan, BruteForce (FTP-/SSH-Patator), WebAttack (brute force, XSS, SQL injection) and Bot. Infiltration (36 flows) and Heartbleed (11 flows) were removed because they are too small to learn or evaluate. Classes are integer-encoded.
- **Normalisation:** 68 features remain. Because they span many orders of magnitude (bytes/s up to ~10^9) and a few contain negatives (Init_Win_bytes = −1), a signed log transform $\text{sign}(x)\cdot\log(1+|x|)$ is applied, followed by z-score standardisation fitted on the training set only, and clipping to ±10.
- **Split:** a stratified 70 / 15 / 15 train / validation / test split (seed 42), shared by every model. Validation and test keep the natural class distribution, so reported metrics reflect realistic traffic.
- **Imbalance handling (training set only):** majority classes are capped at 120,000 flows by random undersampling, and the loss uses inverse-square-root class weights. No synthetic augmentation is used, so that no artificial flows enter the comparison.

Notable finding from EDA. De-duplication after removing Destination Port shrinks PortScan from 158,930 to 1,956 flows: a port scan sends the same tiny probe to many ports, so the flows differ only in the port number. Keeping these copies would place identical vectors in both train and test and inflate scores. Only 270 of ~2.23 million unique feature vectors carry conflicting labels (negligible). The resulting class distribution is shown below (Benign : Bot ≈ 1,319 : 1).

Table I. Class counts after cleaning and in each split

Class	After cleaning	Train (capped)	Validation	Test
Benign	1,896,672	120,000	284,501	284,501
DoS	193,745	120,000	29,062	29,062
DDoS	128,014	89,610	19,202	19,202
PortScan	1,956	1,369	293	294
BruteForce	9,150	6,405	1,373	1,372
WebAttack	2,143	1,500	321	322
Bot	1,437	1,006	216	215
Total	2,233,117	339,890	334,968	334,968

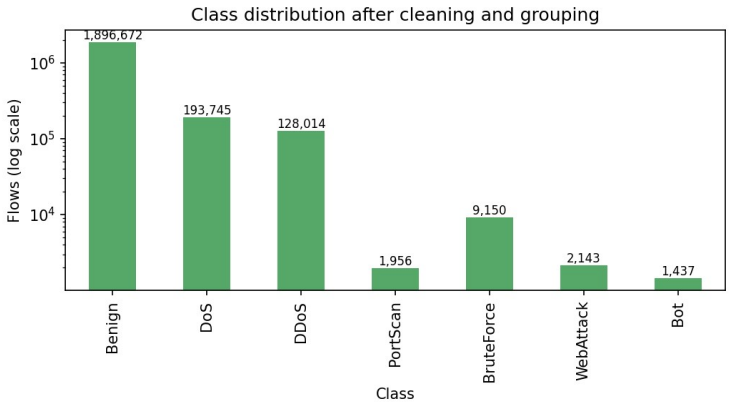


Fig. 1. Class distribution after cleaning and grouping (log scale).

3. Models Implemented So Far (minimum 2 of the proposed 3–4)

Model	Owner (Member)	Status	Preliminary Metric (test)	Notes
MLP (baseline)	Joint (Aditi Arun & Anushka Thakur)	Implemented; preliminary run done	Acc 98.68%, macro-F1 0.818	1 epoch, untuned; 60k params
1D CNN	Aditi Arun	Implemented; preliminary run done	Acc 99.12%, macro-F1 0.838	1 epoch, untuned; 40k params
LSTM	Anushka Thakur	Implemented; first training run in progress	–	~5 min/epoch on CPU; architecture being slimmed
Transformer	Joint (Aditi Arun & Anushka Thakur)	Architecture coded; not yet trained	–	Scheduled per work plan (05/10)

Architectures. The **MLP** baseline has three hidden layers (256-128-64) with batch normalisation, ReLU and dropout 0.2. The **1D CNN** treats the 68-feature vector as a one-channel sequence: three convolution blocks (32/64/128 filters, kernel 3, batch norm, ReLU, max-pooling) followed by global average pooling and a dense head, so its filters see windows of neighbouring features. The **LSTM** reads the features as a 68-step sequence (value + learned feature-identity embedding) with a bidirectional LSTM. The **Transformer** follows FT-Transformer [9]: each feature becomes a token, a [CLS] token is prepended, and self-attention relates any feature to any other regardless of column order.

Common training protocol. All models use the same splits, cross-entropy with class weights, AdamW (learning rate 10^{-3}, weight decay 10^{-4}), batch size 512, gradient clipping at 1.0 and seed 42 (src/train.py). The preliminary runs below are deliberately **unoptimised**: a single training epoch, with no hyperparameter tuning, early stopping or repeated seeds.

Table II. Preliminary test-set results (334,968 flows, 1 training epoch)

Model	Accuracy	Macro P	Macro R	Macro F1	Attack recall	False-pos. rate	F1 PortScan	F1 WebAttack	F1 Bot
MLP	98.68%	0.745	0.956	0.818	99.57%	1.46%	0.810	0.582	0.489
1D CNN	99.12%	0.806	0.951	0.838	97.96%	0.66%	0.878	0.801	0.246

Preliminary observations.

- Both models already separate benign from attack traffic well: the MLP detects 99.57% and the 1D CNN 97.96% of attack flows. Their false-positive rates are 1.46% and 0.66%.
- Accuracy is dominated by the benign class (85% of test flows) and therefore hides weaknesses. Macro-F1 (0.818 vs. 0.838) is limited by the rare classes – PortScan, WebAttack and especially Bot (215 test flows) – where even a small fraction of benign flows misclassified as an attack destroys precision.
- After one epoch the 1D CNN is ahead of the MLP on accuracy, macro-F1 and false-positive rate, while the MLP has the higher attack recall. Whether this reflects the convolutional inductive bias or only training noise must be tested with full training and repeated seeds before drawing conclusions.

4. Individual Contribution Log (to date)

Member Name	Reg. No.	Task(s) Completed	Signature
Aditi Arun	230911108	Dataset acquisition script; cleaning, encoding, normalisation and split pipeline; 1D CNN implementation and preliminary evaluation; joint MLP baseline; literature review (papers [1], [3], [6], [7], [9])	
Anushka Thakur	230911048	EDA and class-distribution analysis; LSTM implementation and first training runs; joint MLP baseline; common training/evaluation script; literature review (papers [2], [4], [5], [8], [10])	

Commit history evidencing individual contributions is available in the GitHub repository linked above.

5. Risk / Plan for Remaining Work

Remaining models to implement, expected challenges, and timeline to Final submission:

Remaining work. Complete LSTM training and evaluation, train the Transformer, then train all four models to convergence under the common protocol, tune the key hyperparameters, and carry out the comparative and error analysis for the Final report.

Table III. Risks and mitigations

Risk / challenge	Impact	Mitigation
No GPU: LSTM (~5 min/epoch) and Transformer (attention over 69 tokens) are slow on CPU	Delayed training; fewer tuning runs	Slimmer recurrent/attention models, cap majority classes in training, run independent experiments in parallel
Severe class imbalance (Benign : Bot ≈ 1,300 : 1)	Unstable rare-class precision; accuracy misleading	Class-weighted loss; report macro-F1, per-class F1 and threshold-free PR-AUC alongside accuracy
Noisy validation signal for rare classes	Early stopping may pick an unrepresentative epoch	Validate on the natural class ratio, use patience-based early stopping, repeat runs over 3 seeds
Dataset artefacts and label noise in CIC-IDS2017 [10]	Over-optimistic or non-generalisable scores	De-duplication, shortcut-feature removal, and discussion as a limitation in the Final report
Differences caused by setup rather than architecture	Invalid comparison	Single shared split, preprocessing, loss, optimiser and stopping rule for all models (src/config.py)

Table IV. Timeline to Final submission

Sl. No.	Component / Work Element / Milestone	Expected Completion
1	LSTM training and preliminary evaluation (Anushka Thakur)	28/09/2026
2	Transformer / attention-based model implementation and training (joint)	05/10/2026
3	Hyperparameter tuning and standardised evaluation of all four models (multiple seeds)	13/10/2026
4	Comparative performance analysis, error analysis and Final report writing	23/10/2026
5	Presentation preparation, viva practice and in-class Final Presentations	26–29/10/2026
6	Final Report and Code Repository Submission	31/10/2026

References

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP), 2018, pp. 108–116.

[2] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, “A deep learning approach to network intrusion detection,” IEEE Trans. Emerg. Topics Comput. Intell., vol. 2, no. 1, pp. 41–50, 2018.

[3] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, “Deep learning approach for intelligent intrusion detection system,” IEEE Access, vol. 7, pp. 41525–41550, 2019.

[4] C. Yin, Y. Zhu, J. Fei, and X. He, “A deep learning approach for intrusion detection using recurrent neural networks,” IEEE Access, vol. 5, pp. 21954–21961, 2017.

[5] J. Kim, J. Kim, H. L. T. Thu, and H. Kim, “Long short term memory recurrent neural network classifier for intrusion detection,” in Proc. Int. Conf. Platform Technol. Service (PlatCon), 2016, pp. 1–5.

[6] W. Wang, Y. Sheng, J. Wang, X. Zeng, X. Ye, Y. Huang, and M. Zhu, “HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection,” IEEE Access, vol. 6, pp. 1792–1806, 2018.

[7] P. Wu and H. Guo, “LuNet: A deep neural network for network intrusion detection,” in Proc. IEEE Symp. Series Comput. Intell. (SSCI), 2019, pp. 617–624.

[8] Y. Yang, K. Zheng, C. Wu, X. Niu, and Y. Yang, “Building an effective intrusion detection system using the modified density peak clustering algorithm and deep belief networks,” Appl. Sci., vol. 9, no. 2, art. 238, 2019.

[9] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, “Revisiting deep learning models for tabular data,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2021, pp. 18932–18943.

[10] G. Engelen, V. Rimmer, and W. Joosen, “Troubleshooting an intrusion detection dataset: the CICIDS2017 case study,” in Proc. IEEE Secur. Privacy Workshops (SPW), 2021, pp. 7–12.